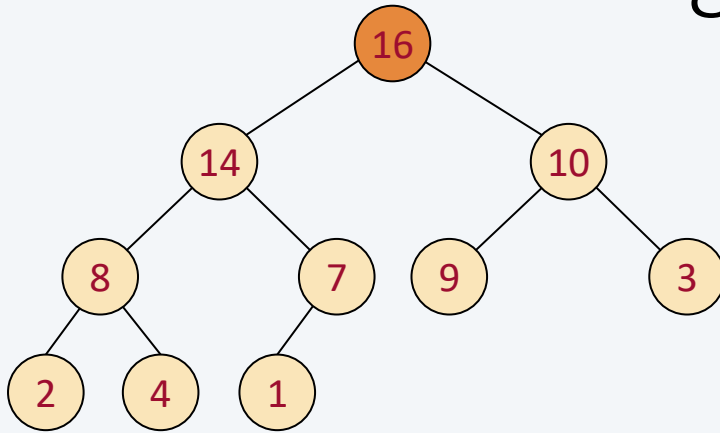


8 Heap-Datenstruktur und -Sortieren

8.1 Heap-Datenstruktur

8.2 Heap-Sortieren



Wir unterscheiden verschiedene **Arten von Behältern**, die jeweils zwei grundlegende Operationen unterstützen: das Einfügen (*insert*) und das Entfernen (*remove*) von Elementen.

Entscheidend ist hierbei, welches Element beim Entfernen gewählt wird:

- **Stack**. Entfernt wird immer das **zuletzt hinzugefügte** Element (Last-In, First-Out – LIFO).
- **Queue**. Hier wird das **zuerst hinzugefügte** Element entfernt (First-In, First-Out – FIFO).
- **Priority Queue**. In einer Priority Queue wird das Element mit dem **größten Wert** (oder alternativ mit dem kleinsten) entfernt. (Wert = Priorität)

Beispiel: Auf einem Server werden die Ausführungszeiten einer Software protokolliert. Aus einem Protokoll mit n Einträgen (wobei $n > 10^9$) sollen die m schnellsten Ausführungszeiten ermittelt werden.

Es gibt nicht genügend Speicher, um alle Daten (Ausführungszeiten) zu sortieren.

Lösung mit Priority Queue:

1. Aufbauen der Datenstruktur

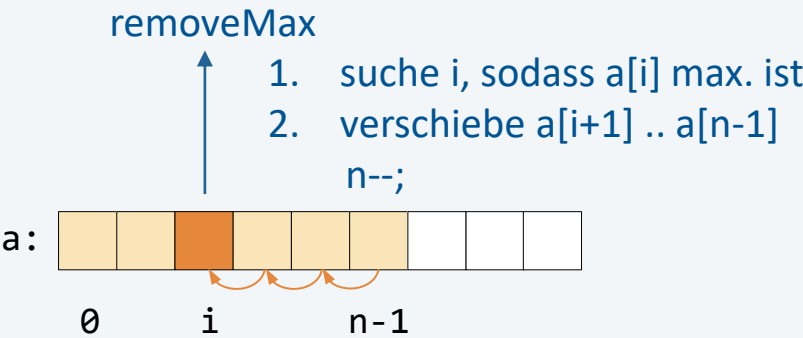
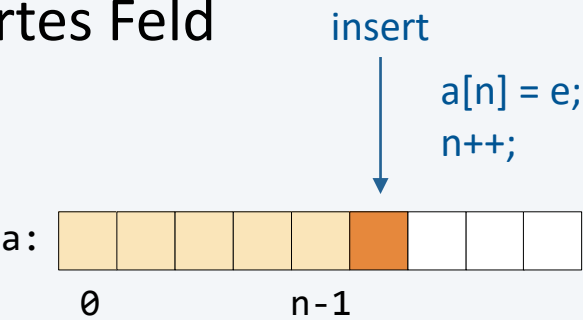
```
int m = 1000;
MaxPriorityQueue pq = new MaxPriorityQueue(m + 1);
for (int e : data from file) {
    pq.insert(e);
    if (pq.size() > m) {
        pq.removeMax();
    }
}
```

sequenzieller Datenstrom
aus Protokolldatei

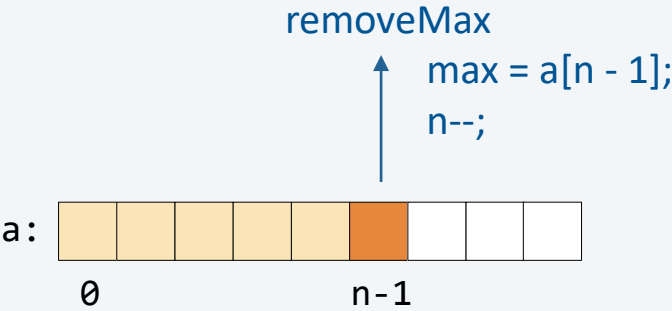
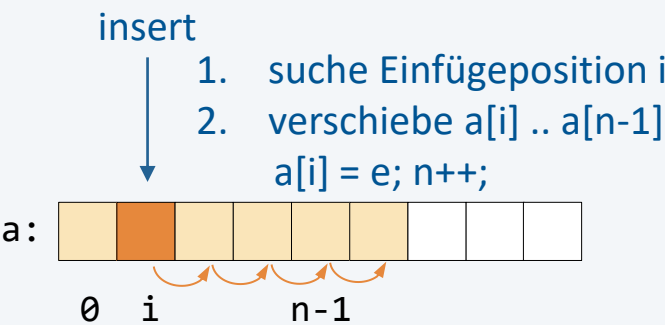
2. Ausgabe

```
while (pq.size() > 0) {
    int e = pq.removeMax();
    IO.println(e);
}
```

Unsortiertes Feld



Sortiertes Feld



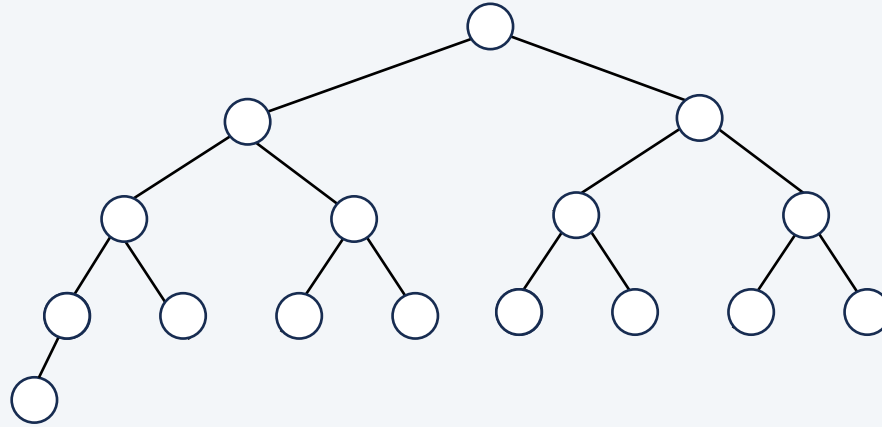
Implementierung	insert	removeMax
Feld (unsortiert)	$O(1)$	$O(n)$
Feld (sortiert)	$O(n)$	$O(1)$
Ziel	$O(\log n)$	$O(\log n)$

Mögliche Implementierungen: Feld

Ausführungszeiten für $n = 10000000$ (gemessen auf DELL Notebook 2025)

m	Feld	Feld (sortiert)	Ziel
1000	1712 ms	1360 ms	179 ms
2000	3286 ms	2573 ms	197 ms
3000	4748 ms	4616 ms	210 ms
4000	6399 ms	5209 ms	246 ms
5000	8993 ms	7023 ms	249 ms
6000	9763 ms	7464 ms	235 ms
7000	11080 ms	9262 ms	232 ms
8000	13172 ms	10229 ms	256 ms
9000	14396 ms	11252 ms	313 ms
10000	17551 ms	12727 ms	276 ms

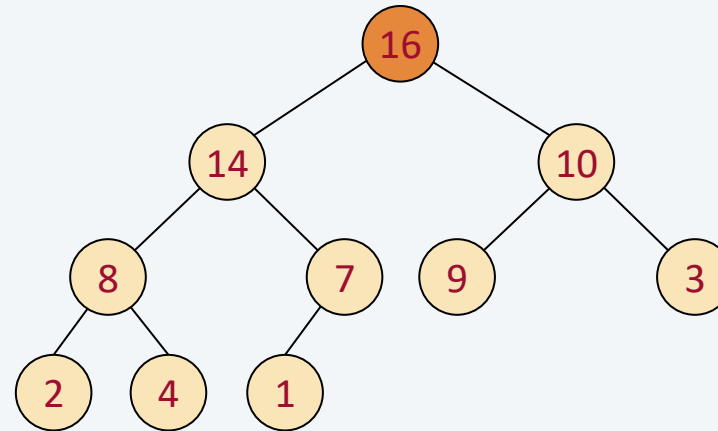
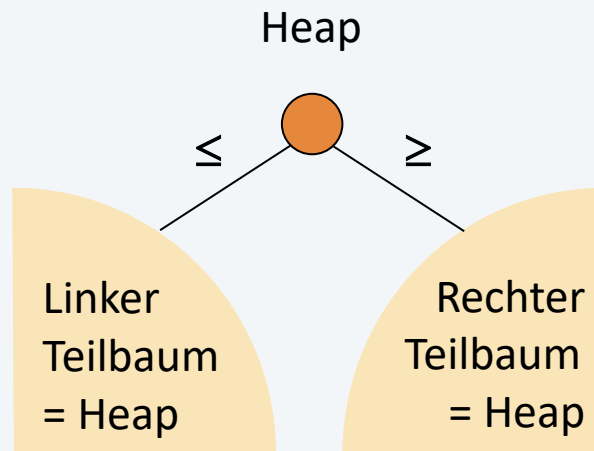
- ein **kompletter Binärbaum** ist in allen Ebenen bis auf die letzte vollständig



kompletter Binärbaum mit $n = 16$ Knoten und der Höhe 4

- er hat in der k -ten Ebene (von $k = 0$ bis zur vorletzten) 2^k Knoten
- nur in der untersten Ebene dürfen „rechts“ Knoten fehlen
- die Höhe eines kompletten Binärbaums mit n Knoten ist $\lfloor \log_2 n \rfloor$

- ein (binärer) Heap ist ein Binärbaum, der in der Wurzel den größten Wert enthält
- wenn diese Eigenschaft auch auf die Wurzeln aller Teilbäume zutrifft, dann ist der Baum ein binärer Heap

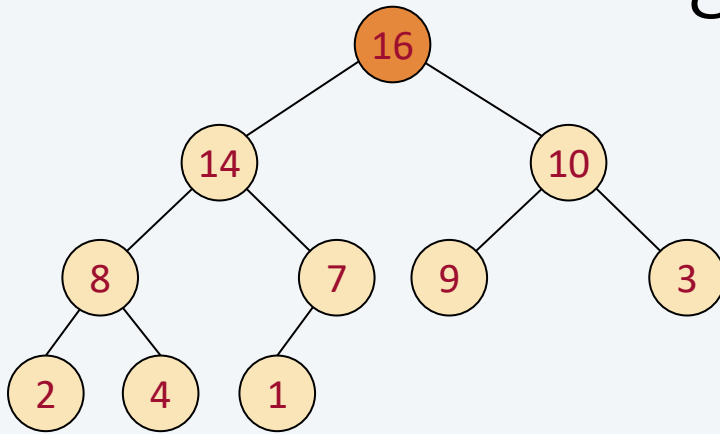


- zwei speziellen Operationen: Einfügen und Entfernen des größten Elements
- Suchen nach Elementen ist keine Operation!

8 Heap-Datenstruktur und -Sortieren

8.1 Heap-Datenstruktur

8.2 Heap-Sortieren



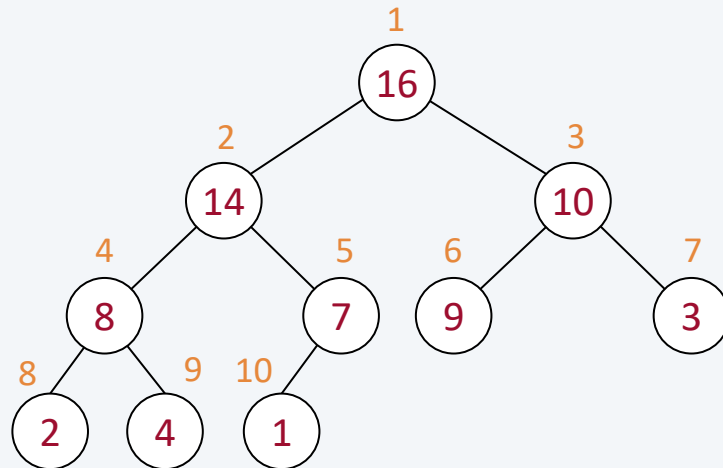
Heap in der Realisierungsform Feld

Ein kompletter Binärbaum kann auch **in Form eines Felds** realisiert werden.

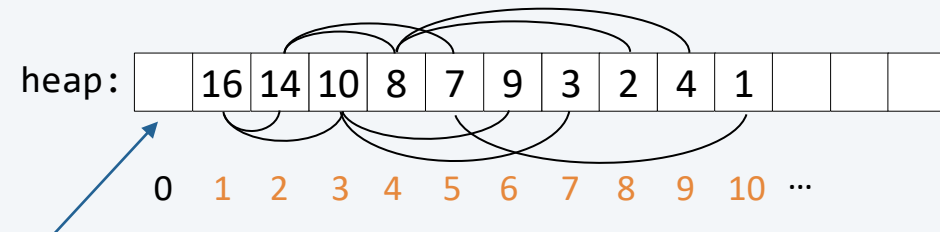
Die Werte in einem Feld $\text{heap}[1:n]$ bilden einen Heap, wenn $\text{heap}[1]$ die Wurzel ist und für jedes Element an der Stelle i gilt, dass

- sich sein linker Nachfolger an der Position $2i$ befindet
- sich sein rechter Nachfolger an der Position $2i + 1$ befindet
- sein Vorgänger an der Position $i/2$ zu finden ist

Beispiel



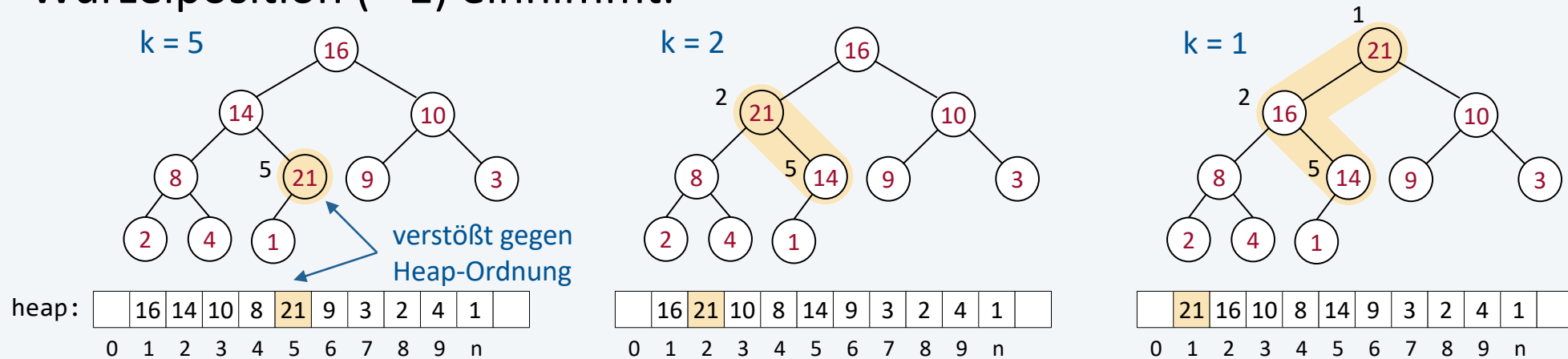
$\text{Left}(i) = 2i$
 $\text{Right}(i) = 2i + 1$
 $\text{Parent}(i) = i / 2$
 für $i > 1$: $\text{heap}[\text{Parent}(i)] \geq \text{heap}[i]$



Position 0 wird nicht genutzt

Problem. Ein Schlüsselwert ist größer als der Schlüsselwert des Vorgängers.

Lösungsidee. Das Element wird so lange mit seinem Vorgänger vertauscht, bis es einen Vorgänger mit einem größeren Schlüsselwert hat oder selbst die Wurzelposition (= 1) einnimmt.



Methode

```
private void swim(int k) {
    while (k > 1 && less(k / 2, k)) {
        swap(k / 2, k);
        k = k / 2;
    }
}
```

Vorgängerknoten von Knoten an der Position k ist an der Position $k / 2$

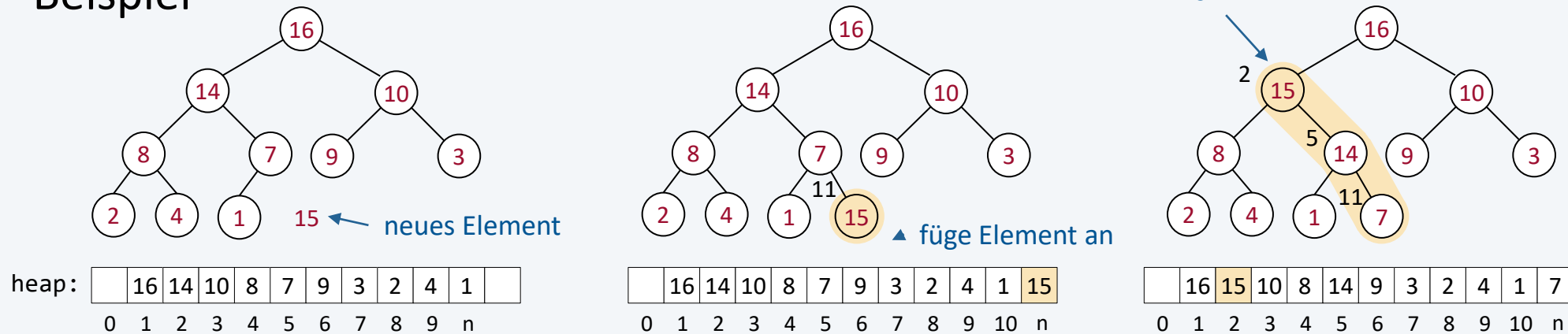
less(int i, int j) vergleicht $\text{heap}[i] < \text{heap}[j]$

swap(int i, int j) vertauscht $\text{heap}[i]$ mit $\text{heap}[j]$

Lösungsidee

- füge neues Element am Ende (an Position $n + 1$) ein
- nutze Algorithmus swim, um Heap-Ordnung wieder herzustellen

Beispiel



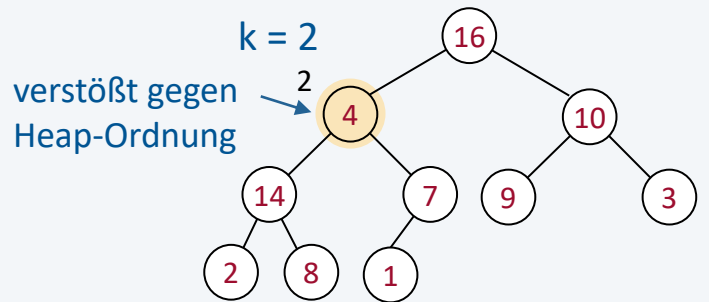
Methode (ohne zu prüfen, ob Heap voll ist)

```
public void insert(int e) {  
    n++;  
    heap[n] = e;  
    swim(n);  
}
```

Problem. Ein Schlüsselwert ist kleiner als einer (oder beider) seiner Nachfolger.

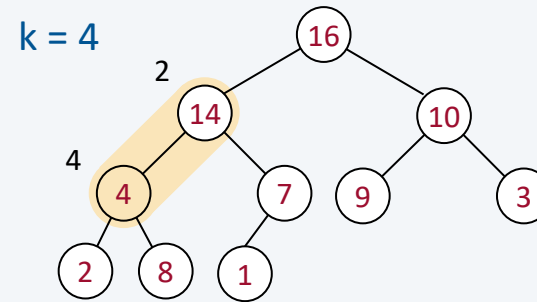
Lösungsidee. Das Element wird so lange mit jenem seiner beiden Nachfolger vertauscht, der den größeren Schlüsselwert enthält, bis kein Nachfolger mit einem größeren Schlüsselwert mehr existiert.

Beispiel

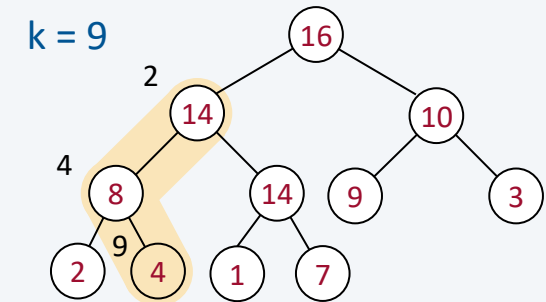


heap:

	16	4	10	14	7	9	3	2	8	1	
0	1	2	3	4	5	6	7	8	9	n	



	16	14	10	4	7	9	3	2	8	1	
0	1	2	3	4	5	6	7	8	9	n	

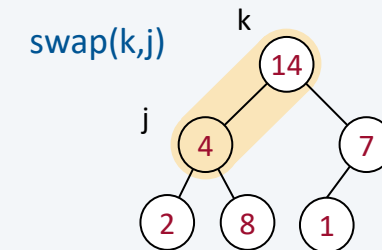
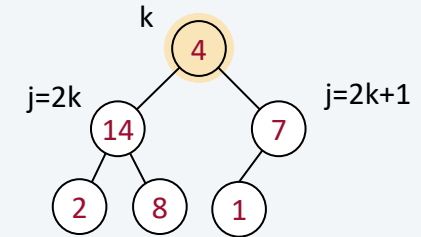


	16	14	10	8	7	9	3	2	4	1	
0	1	2	3	4	5	6	7	8	9	n	

Methode

```
private void sink(int k) {  
    while (2 * k <= n) {  
        int j = 2 * k;  
        if (j < n && less(j, j + 1)) {  
            j++;  
        }  
        if (!less(k, j)) {  
            break; ← Position gefunden  
        }  
        swap(k, j); ← tausche Schlüssel  
        k = j;  
    }  
}
```

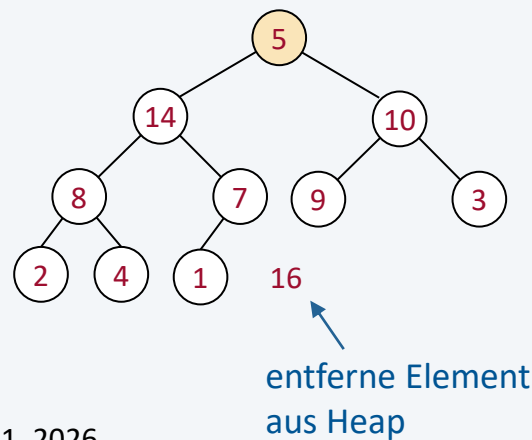
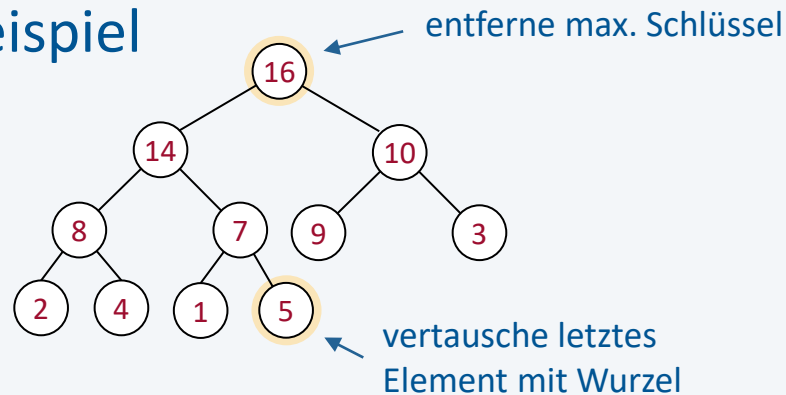
j ... Position des
größeren Nachfolgers
 $j = 2k$ oder $j = 2k + 1$



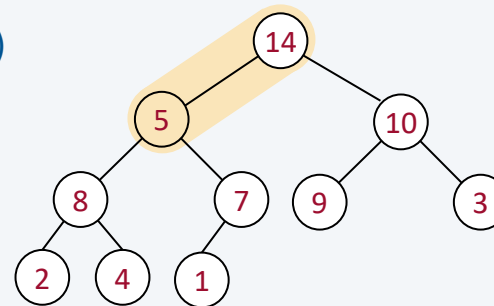
Lösungsidee

- vertausche Schlüssel in der Wurzel mit dem Element am Ende (Position n)
- nutze Algorithmus sink, um Heap-Ordnung wieder herzustellen

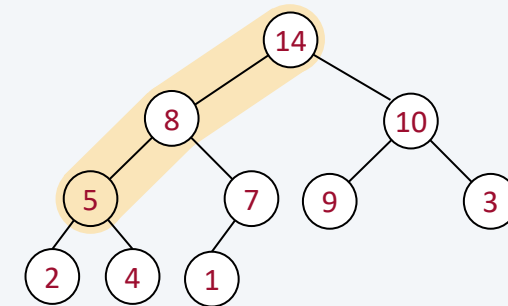
Beispiel



sink(1)



sink(2)



Methode

```
public int removeMax() {  
    int max = heap[1];  
    swap(1, n);  
    n--;  
    sink(1);  
    return max;  
}
```

```
public class MaxPriorityQueue {
    private int[] heap;
    private int n;

    public MaxPriorityQueue(int capacity) {
        this.heap = new int[capacity + 1];
        this.n = 0;
    }
    public      int size() { return n; }
    public      void insert(int e) { ... }
    public      int removeMax() { ... }
    private     void sink(int k) { ... }
    private     void swim(int k) { ... }
    private     boolean less(int i, int j) {
        return heap[i] < heap[j];
    }
    private     void swap(int i, int j) {
        int t = heap[i]; heap[i] = heap[j]; heap[j] = t;
    }
}
```


Das Feld repräsentiert einen kompletten Binärbaum

- die Höhe eines kompletten Binärbaums mit n Knoten ist $\lfloor \log_2 n \rfloor$

Einfügen (`insert`, `swim`)

- das neue Element wandert von der untersten Ebene höchstens bis zur Ebene 0 (der Wurzelposition mit Index 1)
- der Algorithmus `swim` und damit `insert` hat die asymptotische Laufzeitkomplexität $O(\log n)$

Entfernen des Maximums (`removeMax`, `sink`)

- das Element an der Wurzelposition wandert höchstens bis zur untersten Ebene
- der Algorithmus `sink` und damit `removeMax` hat die asymptotische Laufzeitkomplexität $O(\log n)$

Fehlerbehandlung

- insert: wenn das Feld voll ist, kann man ein größeres Feld erzeugen und die Elemente kopieren
- removeMax: Vorbedingung $n \geq 1$ prüfen

MinPriorityQueue

- ersetze less durch greater

Schlüssel sind unveränderlich

- Annahme: Schlüsselwerte in der Heap-Datenstruktur werden nicht verändert
- Tipp: verwende unveränderliche Schlüsselwerte z.B. int, String

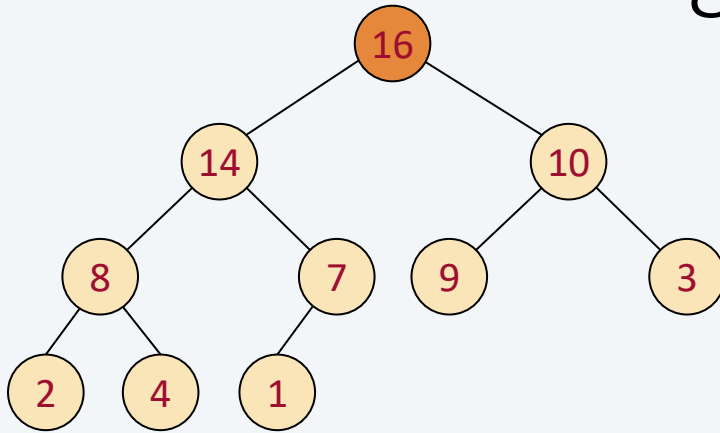
```
private void resize() {  
    int[] copy = new int[2 * heap.length];  
    for (int i = 0; i < heap.length; i++) {  
        copy[i] = heap[i];  
    }  
    heap = copy;  
}
```

```
private void resize() {  
    heap = Arrays.copyOf(heap, 2*heap.length);  
}
```

8 Heap-Datenstruktur und -Sortieren

8.1 Heap-Datenstruktur

8.2 Heap-Sortieren



- Williams und Floyd haben auf Basis der Heap-Datenstruktur ein effizientes Sortierverfahren entwickelt
- das Laufzeitverhalten kommt nahe an das von Quicksort
- dafür ohne Risiko einer quadratischen Laufzeitkomplexität (Degeneration)

Schnittstelle u. Verwendung

```
public class Heapsort {  
    public static void sort(int[] a) {...}  
}
```

```
int[] a = new int[...];  
for (int i = 0; i < a.length; i++) {  
    a[i] = (int)(n * Math.random());  
}  
Heapsort.sort(a);
```



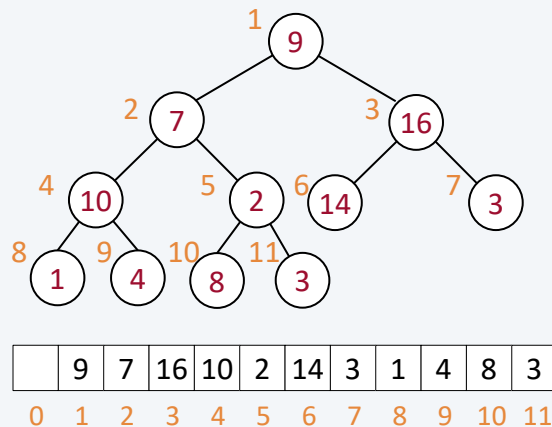
Floyd, R. W.: Algorithm 245, treesort 3;
Communications of the ACM, Vol. 7, No. 12, 1964

Williams, J. W. J.: Algorithm 232: Heapsort;
Communications of the ACM, Vol. 7, No. 6, 1964

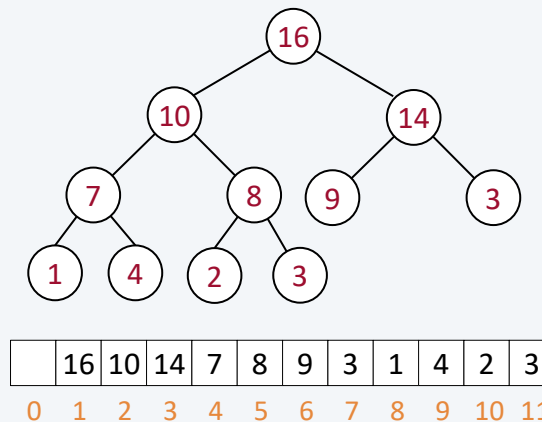
Lösungsidee

- das Feld mit den Schlüsselwerten wird als kompletter Binärbaum betrachtet
- Phase 1: erzeuge Max-Heap mit allen Schlüsselwerten im Feld
- Phase 2: entferne fortlaufend das Maximum aus dem Max-Heap

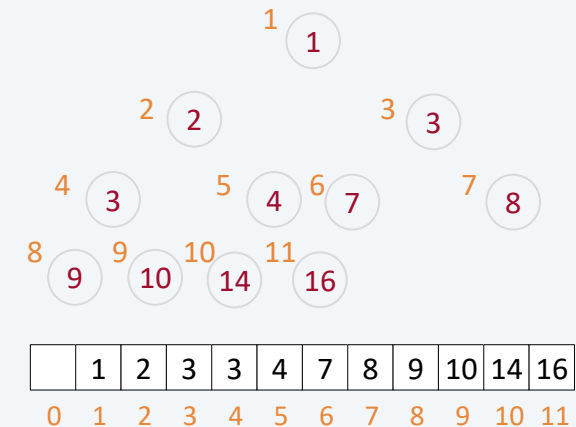
Schlüsselwerte in beliebiger Reihenfolge



Phase 1: erzeuge Max-Heap (in place)



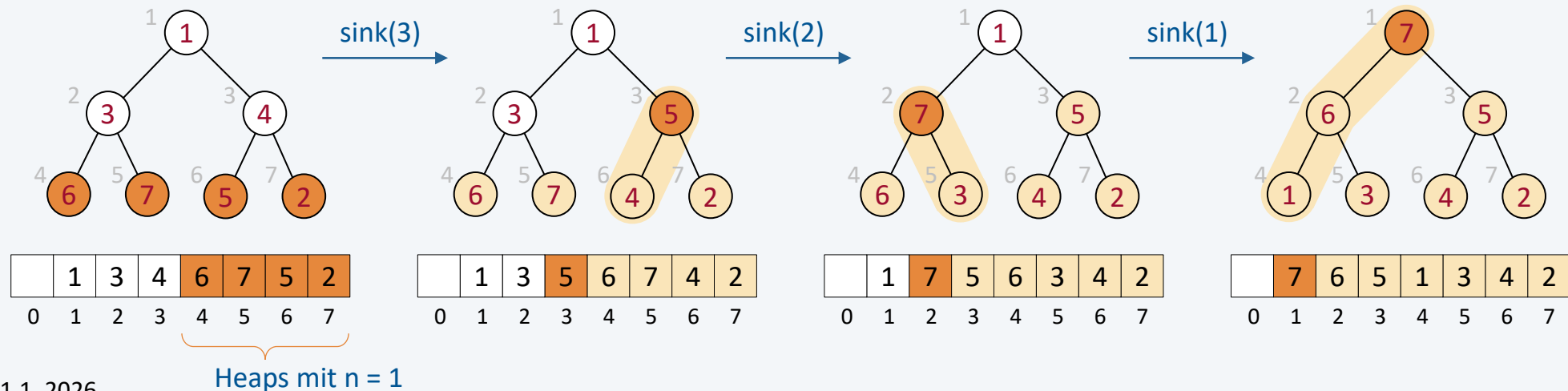
Phase 2: sortiertes Feld



Phase 1 „Aufbauphase“

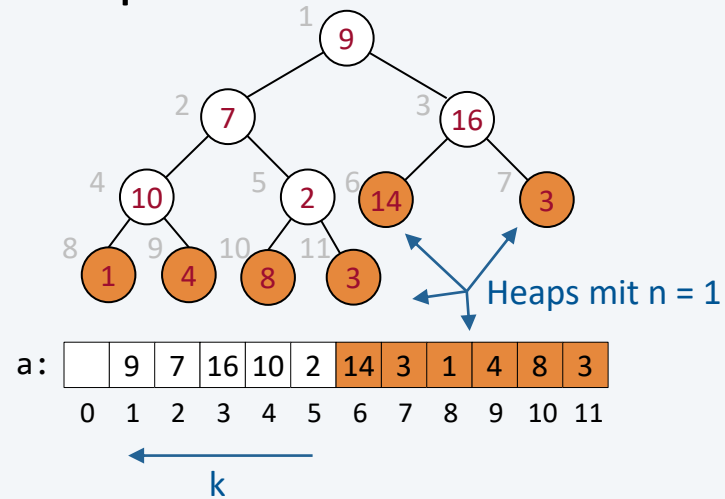
Lösungsidee

- jedes Element in der hinteren Hälfte des Felds wird als Heap betrachtet, der nur aus einem Element besteht
- die Elemente in der vorderen Hälfte werden mittels sink in die Heaps am hinteren Ende eingefügt
- dabei wächst die Größe der Heaps und es verringert sich gleichzeitig ihre Anzahl; zum Schluss dieser Phase bleibt ein einziger Heap übrig, der alle n Elemente enthält



Phase 1 „Aufbauphase“

Beispiel



Lösung für Aufbauphase

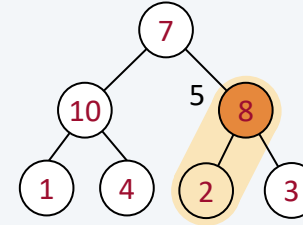
■ a ... zu sortierendes Feld

```
int n = a.length;
for (int k = n / 2; k >= 1; k--) {
    sink(a, k, n);
}
```

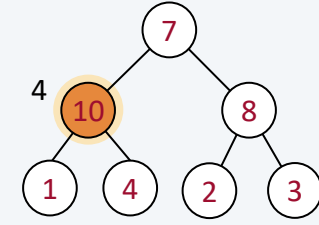
Implementierung wie auf Seite 12
jedoch andere Schnittstelle

`void sink(int[] a, int k, int n)`

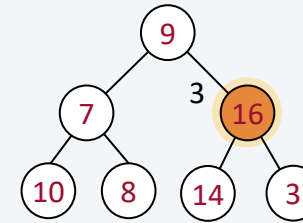
sink(a, 5, 11)



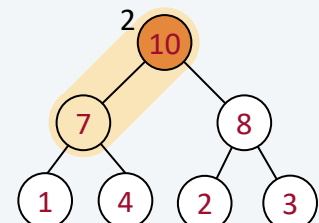
sink(a, 4, 11)



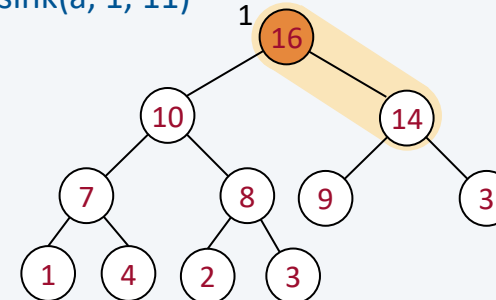
sink(a, 3, 11)



sink(a, 2, 11)

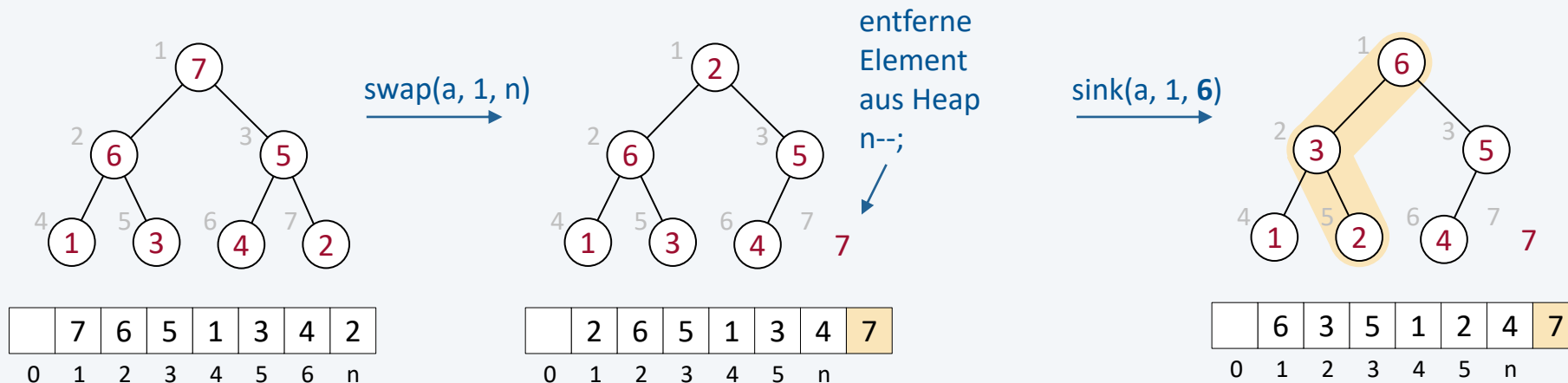


sink(a, 1, 11)



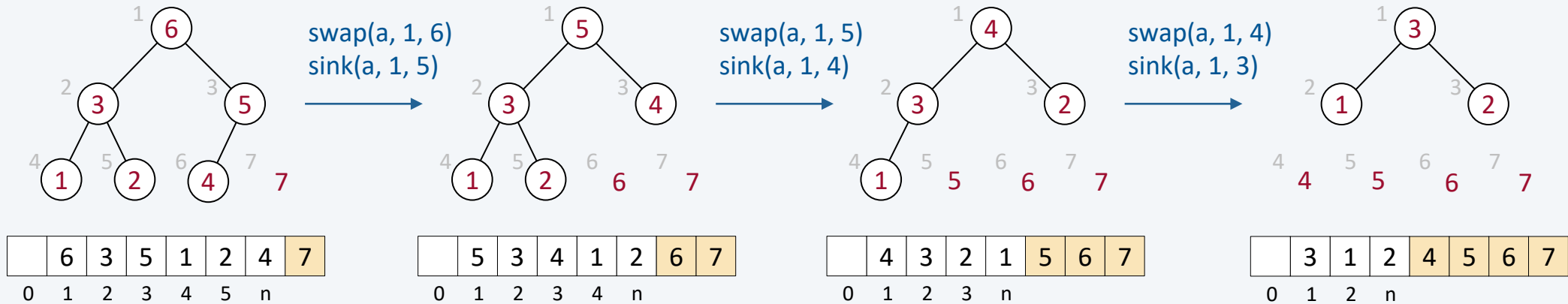
Lösungsidee. Entferne fortlaufend das Maximum aus dem Max-Heap.

- das sortierte Feld wird von hinten her erstellt
- es werden jeweils die Elemente an den Positionen 1 und n vertauscht dadurch gelangt das größte Element (Wurzel) „nach hinten“ an die Position n und das bisher letzte Element des Heaps gelangt nach vorne an die Wurzelposition
- der Schlüssel in der Wurzel wird mittels sink in den um ein Element verkleinerten Heap eingereiht



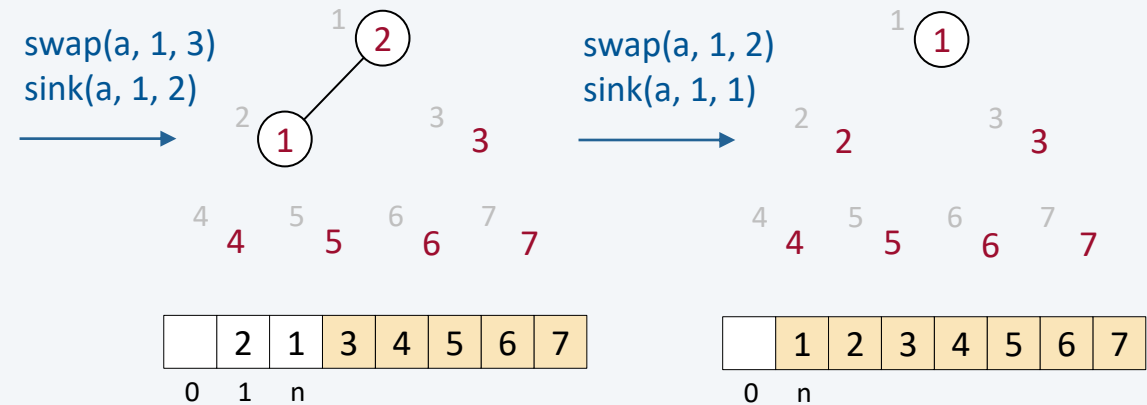
Phase 2

Beispiel



Lösung

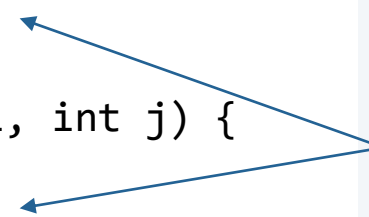
```
while (n > 1) {  
    swap(a, 1, n);  
    n--;  
    sink(a, 1, n);  
}
```



Algorithmus

```
public static void sort(int[] a) {  
    int n = a.length;  
    for (int k = n / 2; k >= 1; k--) {  
        sink(a, k, n);  
    }  
    while (n > 1) {  
        swap(a, 1, n);  
        n--;  
        sink(a, 1, n);  
    }  
}
```

```
private static boolean less(int[] a, int i, int j) {  
    return a[i - 1] < a[j - 1];  
}  
private static void swap(int[] a, int i, int j) {  
    int t = a[i - 1];  
    a[i - 1] = a[j - 1];  
    a[j - 1] = t;  
}
```



Trick: Index jeweils um Wert 1 verringert, um Feld von 0 .. $n - 1$ zu sortieren.

Beispiel 1 (Zustand jeweils nach sink)

Phase 1

n	k		1	2	3	4	5	6	7	8	9	10	11
			9	7	16	10	2	14	3	1	4	8	3
11	5		9	7	16	10	8	14	3	1	4	2	3
11	4		9	7	16	10	8	14	3	1	4	2	3
11	3		9	7	16	10	8	14	3	1	4	2	3
11	2		9	10	16	7	8	14	3	1	4	2	3
11	1		16	10	14	7	8	9	3	1	4	2	3
			16	10	14	7	8	9	3	1	4	2	3

Heap-Ordnung

Phase 2

10	1		14	10	9	7	8	3	3	1	4	2	16
9	1		10	8	9	7	2	3	3	1	4	14	16
8	1		9	8	4	7	2	3	3	1	10	14	16
7	1		8	7	4	1	2	3	3	9	10	14	16
6	1		7	3	4	1	2	3	8	9	10	14	16
5	1		4	3	3	1	2	7	8	9	10	14	16
4	1		3	2	3	1	4	7	8	9	10	14	16
3	1		3	2	1	3	4	7	8	9	10	14	16
2	1		2	1	3	3	4	7	8	9	10	14	16
1	1		1	2	3	3	4	7	8	9	10	14	16

sortiertes Feld

Beispiel 2 (Zustand jeweils nach sink)

Phase 1

n	k		1	2	3	4	5	6	7	8	9	10	11
			16	14	10	9	8	7	4	3	3	2	1
11	5		16	14	10	9	8	7	4	3	3	2	1
11	4		16	14	10	9	8	7	4	3	3	2	1
11	3		16	14	10	9	8	7	4	3	3	2	1
11	2		16	14	10	9	8	7	4	3	3	2	1
11	1		16	14	10	9	8	7	4	3	3	2	1
			16	14	10	9	8	7	4	3	3	2	1

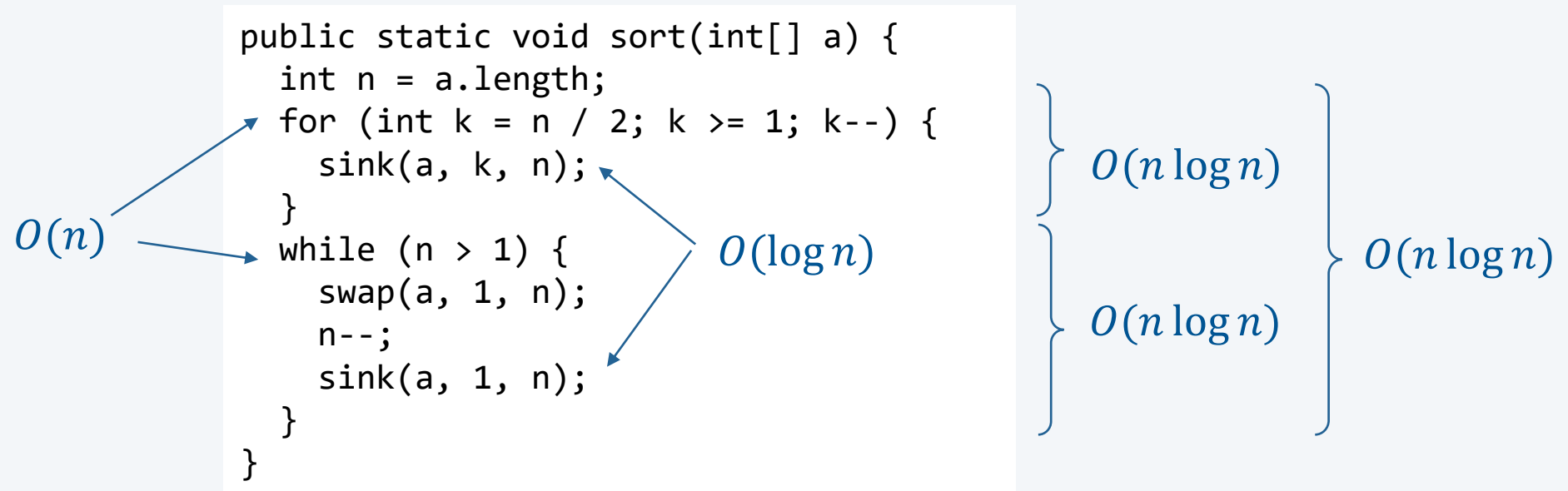
Heap-Ordnung

Phase 2

10	1		14	9	10	3	8	7	4	1	3	2	16
9	1		10	9	7	3	8	2	4	1	3	14	16
8	1		9	8	7	3	3	2	4	1	10	14	16
7	1		8	3	7	1	3	2	4	9	10	14	16
6	1		7	3	4	1	3	2	8	9	10	14	16
5	1		4	3	2	1	3	7	8	9	10	14	16
4	1		3	3	2	1	4	7	8	9	10	14	16
3	1		3	1	2	3	4	7	8	9	10	14	16
2	1		2	1	3	3	4	7	8	9	10	14	16
1	1		1	2	3	3	4	7	8	9	10	14	16

sortiertes Feld

- in beiden Phasen kommt der Hilfsalgorithmus `sink` zur Anwendung
- der Algorithmus `sink` hat die asymptotische Laufzeitkomplexität $O(\log n)$
- deshalb ergibt sich für die asymptotische Laufzeitkomplexität $O(n \log n)$



Interne Sortierverfahren mit Laufzeitkomplexität $O(n \log n)$

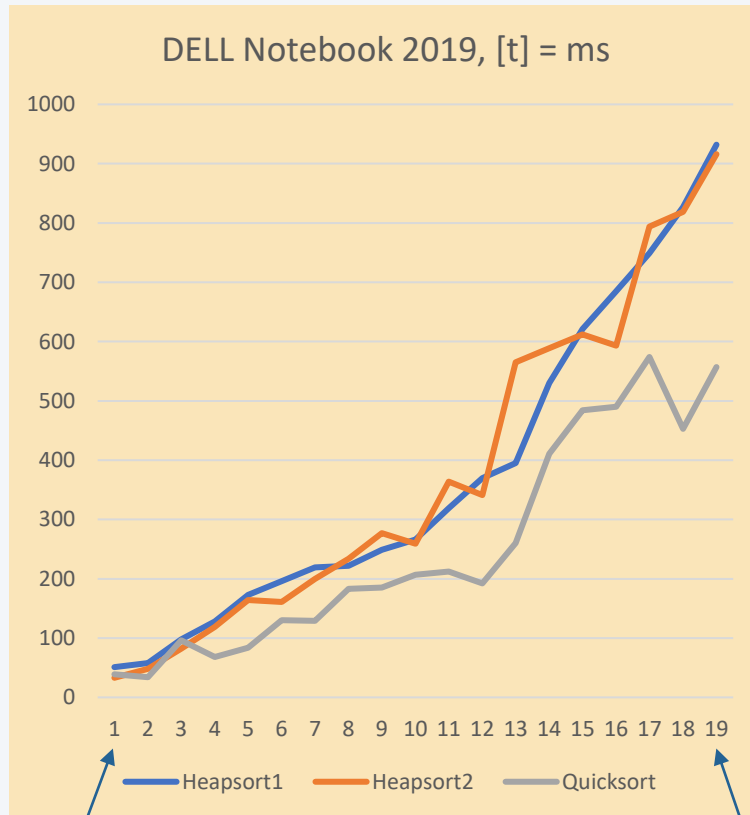
- Quicksort: typisch $O(n \log n)$, aber $O(n^2)$ im ungünstigen Fall
- Mergesort: immer $O(n \log n)$, aber Speicherkomplexität $O(n)$ für merge-Hilfsalgorithmus
- Heapsort: immer $O(n \log n)$

Introsort (z.B. in C++)

- sortiere mit Quicksort
- sortiere mit Heapsort, wenn Rekursionstiefe (am Laufzeitkeller) $2 \log n$ überschreitet
- sortiere mit Insertionsort, wenn $n \leq 16$

Vergleich der Laufzeiten

Laufzeiten t in ms für $n = 100\,000$ bis $1\,900\,000$



$N = 100\,000$

$N = 1\,900\,000$

